

v0.3.2 Evidence / Facts 统一数据层

Codex 分批执行计划书。本文只定义开发、模型接入、本地发布、GitHub 推送和 Vercel 部署流程，不直接执行产品开发。

版本	v0.3.2
定位	把多数据源抓取结果清洗成统一 evidence / facts，为 LLM 和 Serenity 结构化分析打地基。
预计工作量	2-3 个 Codex 工作轮次，属于后续版本的核心地基。
建议分支	codex/v0.3.2-evidence-facts-layer
完成标志	前端不再直接理解各 provider 的临时字段，而是消费统一的研究事实层。

总体链路

多数据源抓取 -> 清洗成 evidence / facts -> 结构化数据类型 -> 喂给大模型 -> 生成模块化分析内容 -> 前端渲染成网页内容。

计划书中的输出均要求证据可追溯、缺失可解释、模型可关闭、部署可回滚。

版本目标

- 建立 EvidenceRecord: 记录来源页面、抓取时间、原文片段、数据源、解析方法和置信度。
- 建立 FactRecord: 把财报、指引、估值、事件、情景输入统一成可验证事实。
- 把 SEC、Yahoo、FMP、东方财富等 provider 输出转换成 facts, 而不是直接拼到 UI。
- 新增 dataQuality 汇总: coverage、freshness、sourceDiversity、warnings。
- 为 LLM 输入增加 token 控制: 只传事实和短证据, 不传整篇原始网页。

主要改动文件

- 新增 src/types/evidence.ts: EvidenceRecord、FactRecord、SourceDescriptor、DataQualityReport。
- 新增 src/lib/research/evidenceStore.ts: 临时内存存储或纯对象聚合, 后续可替换为数据库。
- 新增 src/lib/research/factBuilder.ts: provider 输出到 facts 的转换。
- 新增 src/lib/research/factValidation.ts: 单位、期间、数值区间、来源完整性校验。
- 更新 src/lib/generateResearchCard/mockGenerateResearchCard.ts: 从 facts 组装 card。
- 更新 src/components/generate/GeneratedCardPreview.tsx: 读取统一 dataQuality 和 evidence。

详细开发步骤

- 盘点当前 API 返回的 enhancedEarnings、guidanceData、advancedScenarios、serenityAnalysis 字段, 标记重复和无法追溯的字段。
- 定义 FactRecord.kind: financial_metric、guidance_metric、valuation_metric、event、scenario_input、qualitative_claim。
- 为每个 fact 绑定 evidenceIds, 要求前端任何展示型结论都能追到证据。
- 实现 buildFactsFromEarnings、buildFactsFromGuidance、buildFactsFromScenario、buildFactsFromSerenityInput。
- 实现 normalizeUnits: USD、percent、shares、perShare、ratio 等单位统一。
- 实现 quality report: 缺失源、过期源、低置信度、冲突字段。
- 把旧字段继续保留一版作为兼容输出, 但新 UI 优先读取 facts。

DeepSeek / 大模型接入位置

- 本版本不调用 DeepSeek, 但定义 LLMResearchInput 类型。
- LLMResearchInput 只允许包含 facts、evidence summaries、moduleRequest、language、complianceRules。
- 所有 fact 必须经过 schema 校验后才能进入 LLM 输入, 避免把网页噪声直接喂给模型。

验收标准

- NVDA 生成研究卡时 API 返回 facts 和 evidence 两个顶层字段。
- 公司指引、财报快照、情景输入至少各有一类 FactRecord。
- 每个 FactRecord 至少有一个 evidenceld 或明确 missingReason。
- 旧页面仍能运行，兼容字段没有被突然删除。
- npm.cmd run lint 和 npm.cmd run build 通过。

Codex 首轮执行 Prompt

请继续 v0.3.2: 基于本 PDF 新增 Evidence / Facts 统一数据层，先做类型和转换器，再逐步接入研究卡生成 API。保持旧字段兼容，完成后运行 lint/build，并在 localhost:3000 检查 NVDA 卡片。

通用执行前检查

- 进入项目目录并确认当前分支、远端、工作区差异。不要覆盖用户未提交的改动；若已有改动与本版本相关，先阅读再合并。
- 先阅读 CLAUDE.md、README.md、docs/REAL_DATA_INTEGRATION_PLAN.md，以及本版本涉及的 src/lib、src/components、src/types 文件。
- 每次修改完成后优先运行 npm.cmd run lint 和 npm.cmd run build；Windows PowerShell 下优先使用 npm.cmd。
- 所有外部数据源失败时必须返回可解释的缺失状态，不能让页面直接 500。
- 研究内容必须带 evidenceRef 或 sourceUrl；没有证据时只能显示缺失原因，不能补写事实。

本地发布流程

每个小版本完成后都要先在本地跑通，再决定是否推送。

```
cd "C:\Users\■■■■■■\Desktop\moki-research-card"
npm.cmd install
npm.cmd run lint
npm.cmd run build
npm.cmd run dev -- --hostname 127.0.0.1 --port 3000
# ■■■■■■ http://localhost:3000/ ■■■■■■
```

GitHub 推送流程

建议每个小版本单独分支和单独 commit，用户确认后再进入 main。

```
git status -sb
git checkout -b codex/v0.3.2-short-name
git add -A
git commit -m "feat: v0.3.2 short release title"
git push origin codex/v0.3.2-short-name

# ■■■■■■■■ main ■■■
git checkout main
git pull --ff-only origin main
git merge --ff-only codex/v0.3.2-short-name
git push origin main
```

Vercel 部署流程

- 如果 Vercel 项目已连接 GitHub，推送到 main 后会触发生产部署；预览分支会生成 Preview Deployment。
- 在 Vercel Project Settings - Environment Variables 中配置本版本需要的变量，并同时勾选 Production、Preview、Development。
- 部署后检查 Vercel Build Logs、Function Logs、页面首屏、NVDA 样例、无数据 ticker 样例和移动端布局。
- 生产环境不要暴露任何 API Key 到 NEXT_PUBLIC_ 变量；所有抓取和 LLM 请求必须在服务端执行。

通用回滚流程

- 若本地 lint 或 build 失败，先修复，不进入 GitHub 推送。
- 若 Vercel 部署失败，保留失败部署日志，回退到上一个成功 main commit，或重新推送修复 commit。
- 若外部数据源临时失败，页面必须保留上一版可用的缺失态和数据源说明。
- 若 LLM 输出未通过 schema 校验，回退到确定性摘要，不渲染未经校验的文本。

参考链接

- SEC EDGAR APIs: <https://www.sec.gov/search-filings/edgar-application-programming-interfaces>
- DeepSeek API Docs: <https://api-docs.deepseek.com/>
- Vercel Deployments: <https://vercel.com/docs/deployments>
- Vercel Environment Variables: <https://vercel.com/docs/environment-variables>